

VILNIUS UNIVERSITY

FACULTY OF MATHEMATICS AND INFORMATICS

SOFTWARE ENGINEERING STUDY PROGRAMME

Group Project

Notepad#

Tautvydas Danilevičius

Jaroslav Drobitko

Jonas Grubliauskas

Mantas Kalvinskas

Kipras Vytautas Spirgys

Vilnius
2026

Contents

1 Introduction **3**

2 Definitions **4**

3 Feature list (MoSCoW) **6**

4 Non-functional requirements **8**

5 High level architecture **9**

5.1 Overview 9

5.2 Components 10

5.2.1 Application 10

5.2.2 Editing 11

5.2.3 Configuration 12

5.2.4 Input 13

5.2.5 User Interface 13

5.2.6 Extensions 16

5.2.7 Infrastructure 17

5.3 Data Store 19

5.4 Dependencies 21

5.5 API 22

6 Low fidelity wireframes **25**

7 Data model **26**

7.1 Main components 26

1 Introduction

Notepad# is a Free and Open Source editor (licensed under GPL 3), that is designed to be customisable, simple to use and provides many facilities to make code editing simple, fast and enjoyable. There are many existing editors (VS Code, IntelliJ and others), however they usually are large, complicated pieces of software that are hard to customise, use lots of computer resources and have many features that most users will not use. This editor attempts to solve this problem by putting emphasis on customisability via a built-in plugin system, while being similiarly performant and providing modern features.

2 Definitions

Before continuing, it is important to establish some definitions. Some terminology has existing definitions, but their meaning is further narrowed within this section. Each term below is used with exactly one meaning throughout the document, and the terms in the left column are usually the ones used for these concepts.

- **Repository**: maintains and provides access to a collection of persistent or in-memory domain objects.
- **Controller**: performs operations on domain state.
- **Service**: provides functionality that does not naturally belong to a single domain object.
- **View**: represents domain or application state for presentation.
- **Data Store**: the part of the operating system's filesystem in which the editor keeps its own data (configuration, installed plugins, undo history). Files edited by the user are not part of the Data Store.
- **Buffer**: a data structure for storing text, together with the state needed to edit it. A buffer may be associated with a file (a *file buffer*) or not (a *scratch buffer*). The word "document" may be used as a synonym.
- **Text/character encoding** (`https://en.wikipedia.org/wiki/Character_encoding`): a convention of using a numeric value to represent each character of a writing script. Examples include ASCII, Unicode (UTF-8, UTF-16, etc.) and EBCDIC.
- **Newline (or Line Ending)**: Character or sequence of characters used to signify the end of a line.
- **Character**: a code point. All positions and lengths inside a buffer (cursor positions, selection bounds, rope weights, positions stored in undo history) are counted in characters, independently of the file's encoding. Cursor movement and deletion operate on grapheme clusters, which may consist of several characters.
- **Position**: a non-negative integer offset, in characters, from the start of a buffer.
- **Cursor**: a position in a buffer at which text is inserted and from which editing operations start.
- **Selection**: a range of a buffer between two positions, which editing operations such as copy and delete can act on. Each cursor has at most one selection.
- **Multicursors**: support for more than one cursor within the same editing context.

- **Editing context**: the state of one user's interaction with one buffer: a reference to the buffer, its cursors and selections. A buffer may have several editing contexts, and each pane displays exactly one.
- **Edit**: a single recorded insertion or deletion of text in a buffer. Edits are the entries of the undo and redo history.
- **Editor input**: a platform-independent representation of a user input, such as a key combination or a mouse click.
- **Command**: a named operation that the editor can perform (for example `buffer.save`). A command is identified by a command identifier. This is what was previously called an "editor action".
- **Command Registry**: the collection of all commands known to the editor, built-in and registered by plugins. For each command it stores the identifier, the default key combination (if any) and the operation to perform.
- **Keybind**: an association of a key combination with a command identifier.
- **Pane**: a rectangular UI element that displays one editing context, and therefore one buffer. Also may be referred to as the "text pane" and "Pane View". A pane owns only presentation state (position, size, visible region); cursors and selections belong to its editing context.
- **Focused pane**: the pane that receives editor inputs.
- **Configuration**: the set of user-controlled values (settings, keybinds, enabled plugins) that change the behaviour of the editor. It is stored in the configuration file and described in the Configuration section.
- **Setting**: a named configuration value, identified by a key (for example `fontSize`).
- **Plugin**: a unit of extension code that runs in the Script Runtime and accesses the editor only through the Plugin API.
- **Plugin API**: the set of operations the editor exposes to plugins, provided by the Editor Service.
- **Plugin catalog**: the list of plugins available for installation, hosted remotely and kept only in memory.
- **Installed plugin**: a plugin which exists in the plugins directory of the Data Store.
- **Enabled plugin**: an installed plugin that is currently active.
- **Event**: a notification about a change in the editor, delivered by the Event Bus to the components that subscribed to it.

3 Feature list (MoSCoW)

Here are ideas for the functionality of our text editor:

1. Must:

    - Work on Windows, Linux (at least on a single Linux distribution)
    - Be able to read, write, create, delete files and folders.
    - Be able to read and interpret keyboard inputs.
    - Be able to read and interpret mouse inputs.
    - Support text buffers (see definition above).
    - Support text buffers tied to specific files (for representing the text inside of a file).
    - Be able to edit text within a buffer.
    - Action history/undo-redo behaviour (can be simple, like having a history of changes within a buffer and being able undo/redo them, or maybe something more complex like undotrees)
    - Be able to save all changes made to a buffer, that represents the text of a specific file, to said file.
    - Have UTF-8 as the text encoding for representing text inside of buffers.
    - Be able to copy text from a buffer to a clipboard (or clipboards), and paste text from a clipboard (or clipboards) to a buffer.
    - Be able to have multiple buffers in memory at the same time.
    - Be able to navigate between opened files (files that are in memory).
    - Have cursors, which a user can move and write text with, as the primary way to edit text.
    - Have the ability to select text.
    - Have the ability to manipulate selected text (i.e. copy, delete, etc.).
    - Allow the user to perform all commands using keyboard inputs/shortcuts.
    - Have a graphical user interface (GUI).
    - Support syntax highligthing for multiple file formats.
    - Have keyboard shortcuts. (for example, ctrl-c for copy, ctrl-v for paste, ctrl-z for undo, etc.). **NOTE:** Document defines various shortcuts as examples, but the team may decide what kind of shortcuts are used.
    - Ability to display text.
    - Have UI elements and the ability to display them.

- Have a specific UI element (that will be referred to as a ”text pane” or ”Pane”), that is tied to a buffer and allows a user to perform any of the aforementioned actions within the feature list on the buffer, as well show all the text inside the buffer (for example, by showing the user a specific part of the buffers contents, and allowing to navigate to any other part of the buffer).
- Provide a mechanism for having multiple ”text pane” UI elements within the editor at the same time and be able to navigate to them.
- Provide a mechanism for switching a ”text pane’s” current buffer to another one.
- Be able to create and install plugins that will be visible in a catalog and will apply to all files.
- Have a built-in programming language interpeter.
- Have customisable behaviour, such as custom shortcuts, custom colorschemes or themes, custom fonts, etc.
- Be able to programmatically customise the behaviour of the editor to a degree (example, plugins) by using the built-in programming language interpreter (as an example: Emacs lisp, Neovim’s Lua/Vimscript).

2. Should:

    - Auto complete options.
    - Have a way to search for text within a file, and replace it (search and replace).
    - Autosave feature.

3. Could:

    - Multicursor support.
    - Have a module for Treesitter support (`https://en.wikipedia.org/wiki/Tree-sitter_(parser_generator)` for fast, context dependent code highlighting).
    - Have a module for Language Server Protocol (LSP) support, to reuse existing Language Servers for various languages, granting various features, such as function definition lookup, code errors/warnings without compilation, etc.
    - Have a module for an efficient, built-in way to navigate between files (as an example: VS Code’s file-tree, but I’m thinking of something like Neovim’s telescope.nvim and oil.nvim).
    - Ability to render more complex visual effects, such as image rendering.

4. Won’t:

    - Have a dedicated Terminal User Interface (TUI).

4 Non-functional requirements

Times and memory use below are measured on a computer with a 4-core 64-bit processor, 8 GB of RAM and an SSD.

- The editor runs on 64-bit Windows 10 and 11 and on at least one Linux distribution.
- With no plugins enabled, the main window accepts input within 1 s of starting the editor.
- With no plugins enabled, a typed character is shown within 100 ms, even with a 1 GB file open.
- Files up to 1 GB can be opened and edited. A 1 GB file loads within 5 s, and other buffers can be edited while it loads.
- With one empty buffer the editor uses less than 100 MB of memory. Opening a file adds at most twice the file's size to the editor's memory use.
- If saving fails, the file on disk keeps its previous content.
- A failed file operation is shown to the user and does not close the editor.
- A plugin call longer than 1 s is aborted.

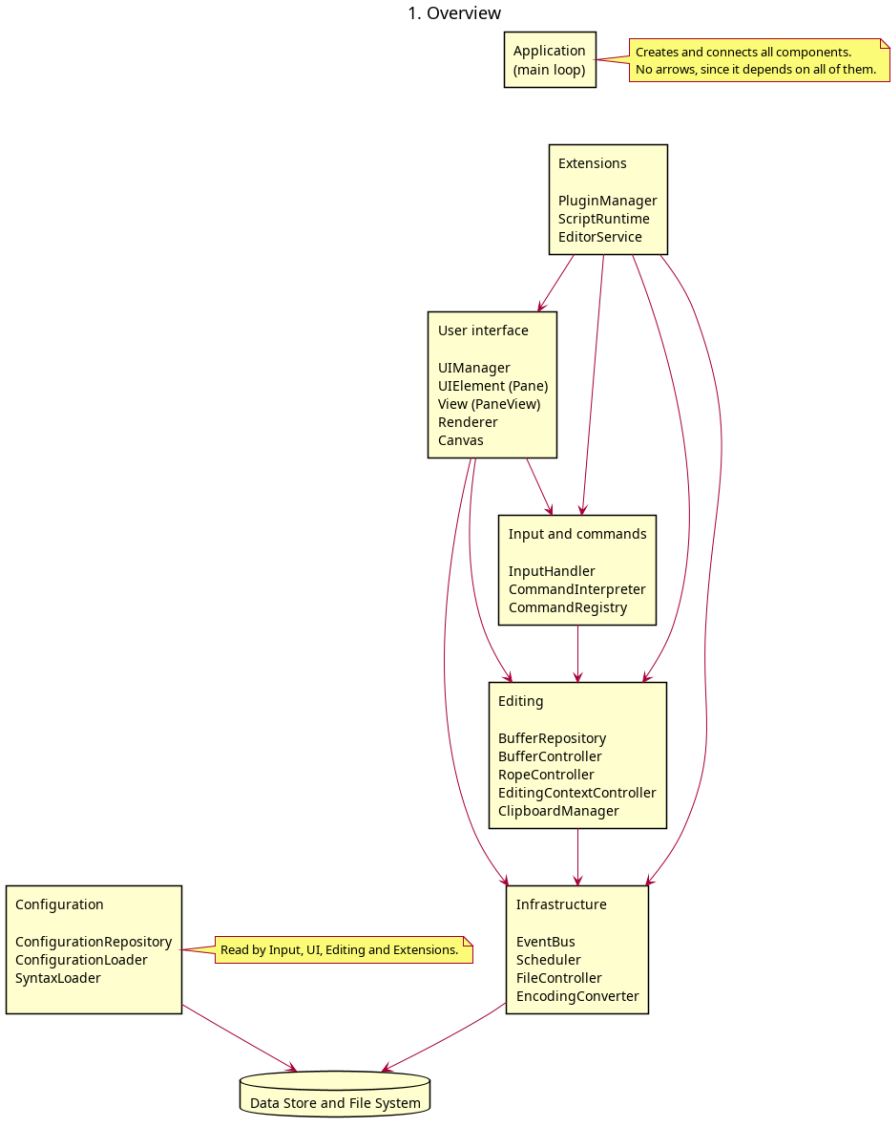
5 High level architecture

5.1 Overview

Notepad# is a single-process desktop program built as a small core with an extension mechanism. The core is organised in layers. The components consist of Controllers, Repositories, etc. Model-View separation is used inside the user interface: Views only draw, UIElements handle input, and the state they present lives in Buffers and EditingContexts.

5.2 Components

The following components constitute the initial logical decomposition of the editor.



1 figure. *System overview (Arrows mean "uses")*

5.2.1 Application

The Application is the entry point of the editor and the component that connects all the others. It creates the other components in dependency order, owns the main loop, and decides where each input goes. It contains no logic of its own.

After startup the Application runs the main loop until the editor is closed. Each iteration:

1. Waits until something needs attention: a platform event, a timer that is due, or a result posted by background work.
2. Handles every platform event that is waiting.
3. Runs the results posted by background work and the timers that are due, through the Scheduler.

4. If the UIManager reports that something has to be redrawn, draws one frame. Many changes within one iteration therefore cause only one redraw.

This component handles inputs as follows:

- Platform events are translated into editor inputs by the InputHandler.
- Keyboard inputs go to the CommandInterpreter first.
- If no keybind matches and the input produces text, the text is delivered to the focused UIElement through the UIManager.
- Mouse inputs go to the UIManager.
- Events about the window itself are handled by the Application (for example, a resize is forwarded to the UIManager, changes of window focus or display scale are published as events or forwarded to the Renderer, etc.)

The Application asks the Renderer for a Canvas, gives it to the UIManager to draw the UIElements, and then tells the Renderer that the frame is complete.

All editor state is changed on the main thread, which is also where commands, event callbacks and plugin code are executed. Work that would block the interface, such as reading a large file, may run in the background, but it communicates with the rest of the editor only through the Scheduler.

5.2.2 Editing

BufferRepository The BufferRepository owns all buffers currently loaded into memory. The repository is responsible for locating, creating, and removing buffers. It does not itself perform the individual text manipulation operations on buffers.

BufferController The BufferController provides operations for directly manipulating the contents and state of Buffers managed by the BufferRepository. The BufferController may depend on the FileController when operations require interaction with the filesystem.

Potential operations include:

- Creating and deleting buffers.
- Reading Buffer contents.
- Inserting, deleting, and replacing text, by delegating to the RopeController.
- Getting a Buffer's file (if exists)
- Associating a Buffer with a file.
- Saving a Buffer to its associated file.
- Reloading a Buffer from its associated file.

RopeController The RopeController performs operations on the Rope that stores the text of a Buffer. It contains the tree logic and nothing else.

Should implement all (necessary) operations associated with the Rope data structure (`https://en.wikipedia.org/wiki/Rope_(data_structure)`).

The maximum leaf size, the balancing rule and the internal string representation are implementation decisions, but they must meet the performance requirements for large files.

ClipboardManager Manages all internal editor clipboards (if any) and interacts with the system clipboard. Internal clipboards may be represented using Buffers.

EditingContextController Has a n:1 relationship with a buffer, basically allowing the same buffer to have multiple EditingContexts. Instead of allowing direct manipulation of a buffer, editing is done through cursors, selections, and other tools. An EditingContext may contain:

- A reference to the currently displayed buffer.
- One or more cursors.
- Selection information.
- Other state describing how the user is interacting with the associated buffer.

Furthermore, potential operations include the following:

- Changing the associated buffer.
- Creating, removing, and moving cursors.
- Obtaining and modifying selections.

5.2.3 Configuration

SyntaxLoader This component is responsible for loading language specific syntax files. Used by the FileController to acquire syntax rules.

ConfigurationRepository The ConfigurationRepository provides access to the configuration of the editor. The repository is responsible for representing and providing access to configuration, while interpretation of configuration for a particular purpose should be performed by the component responsible for that purpose.

ConfigurationLoader The ConfigurationLoader reads and writes the configuration file in the Data Store. At startup it parses and validates the file, builds the Configuration object and hands it to the ConfigurationRepository. When the ConfigurationRepository reports a change, the ConfigurationLoader writes the file. It is the only component that knows the file format (see the Data Store section).

5.2.4 Input

InputHandler The InputHandler translates platform-specifc low-level events, such as keyboard presses, releases, mouse movement and mouse buttons into platform-independent editor input events. The resulting editor input is then passed to other components.

A textual representation such as `<C-w>` may be used for configuration or user-facing purposes, but the internal representation doesn't necessarily have to be textual.

CommandInterpreter The CommandInterpreter converts editor keyboard inputs into editor commands and causes the corresponding operations to be performed.

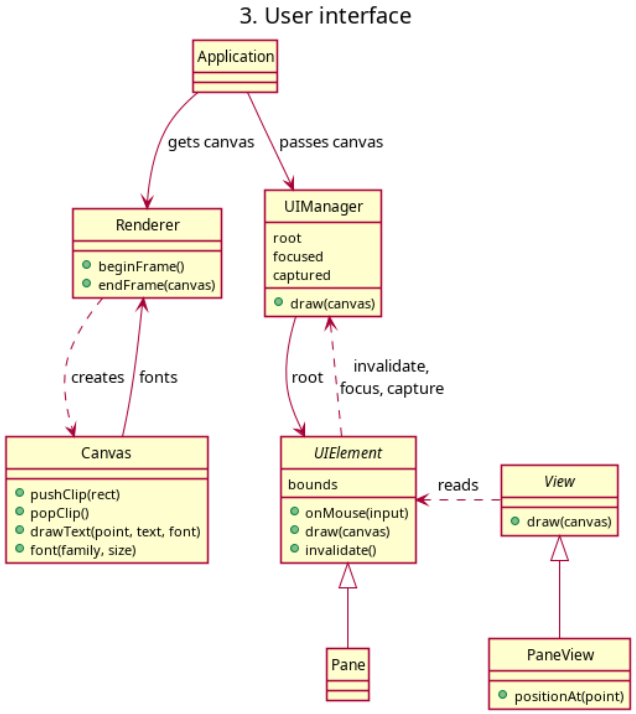
The CommandInterpreter uses the ConfigurationRepository to determine the command associated with a particular input. For example, if `Ctrl+W` is configured to close the current pane, the Command Interpreter will resolve the input to the appropriate command and delegate the operation to the appropriate component.

This separation allows commands to have multiple possible sources. Eventually, commands may be issued by macros, plugins, scripts, or other parts of the application.

CommandRegistry Defines default shortcuts for various commands.

5.2.5 User Interface

The following diagram shows the structure of the UML diagram (the Canvas is created by the Renderer, and then passed to the UIManager and it's constituent UIElements:



2 figure. *User interface class diagram*

UIManager The UIManager holds the tree of UIElements and forwards inputs and drawing to it. It does not draw anything itself, does not know what individual UIElements do, and does not depend on the Renderer.

It is responsible for:

- Holds the root of the UIElement tree. When the window is resized, or an element is added or removed, it asks the tree to recompute the layout.
- Forwarding mouse inputs to the root element. Window coordinates are converted into the coordinates of each element on the way down, so elements never work with window coordinates.
- The focused element: the element that receives text input. At startup this is the first Pane. If the focused element is removed, another focusable element is focused.
- The mouse capture: after an element has consumed a mouse press, it receives all following mouse inputs directly until the element is ”released”, even when the pointer leaves its bounds. This is what makes dragging possible.
- The redraw state: UIElements report that they have to be redrawn, and the UIManager tells the Application whether a frame is needed. When it is asked to draw, it passes the Canvas to the root element.

UIElement A UIElement is a part of the window. The user interface is a tree of UIElements, and UIElement is the base type of everything that is displayed or receives mouse input. Every UIElement has:

- Bounds, relative to its parent. A UIElement works only in its own coordinates, with the origin at its top left corner.
- A parent and zero or more children.
- An View, which draws it.
- Handling of mouse inputs and text input.

It’s functions are as follows.

- By default, UIElements pass mouse input to it’s children, this behaviour may be overriden by subclasses of UIElement.
- Drawing goes from the root to the leaves, passing the Canvas along the way. Each element restricts the Canvas to its own bounds, lets its View draw, then draws its children.
- When the appearance of an element changes, it reports this to its parent, and the report travels up to the UIManager.

- A UIElement uses two mechanisms, for different purposes. When the user does something that should have an effect, the element carries out the operation itself or delegates it to the CommandInterpreter. When an element has to follow a change in the editor, it subscribes to the corresponding events and acts on them.

UIElement has many subclasses. The most prominent example is Pane, which is described in this section, but the rest are not. Examples of UIElements are: Buttons, Labels, Containers, etc.

View A View is only responsible for how an element looks, drawing it. Each element owns at most one View, and the View refers back to its UIElement to read what it needs. Every View draws on the Canvas that it receives for the frame.

Pane A Pane is a UIElement that displays one editing context, and therefore one buffer. It holds presentation state only: its bounds, the part of the buffer that is visible (the first visible line and the horizontal scroll), and a reference to its EditingContext, which holds the cursors and selections. The Pane does not own the EditingContext and does not own the buffer. The Pane turns inputs into operations on other components:

- A mouse press asks its PaneView which position of the buffer is under the pointer, places the cursor there, focuses the Pane and captures the mouse. A double or triple click selects a word or a line.
- While the mouse is captured, mouse movement extends the selection from the point where the press happened to the position under the pointer.
- Scrolling changes the visible part.
- Text input is passed to the BufferController, which changes the buffer at the cursors.

The Pane never changes the text or the cursors itself. It only asks the BufferController and the EditingContextController to do so. A Pane reacts only to events about the buffer it displays.

PaneView The PaneView draws a Pane. It also answers two questions for the Pane: which position of the buffer is under a given point of the pane, and where in the pane a given position is displayed. The first answer is always a position between two characters. These questions are the reverse of drawing, and must agree with it exactly (fonts, tab width, line numbers, scrolling), which is why they are handled by the PaneView and not by the Pane. Measuring text for this purpose is done through the Canvas.

Renderer The Renderer is the editor's interface to the rendering dependency and to the drawing surface of the window. It knows nothing about UIElements, Views or buffers, and depends only on the window.

It is responsible for:

- Creating a Canvas at the start of a frame and displays the result when the frame is completed.

- Measuring the size of text in a given font, and which character lies at a given horizontal distance within a line of text. This is needed outside of frames, for example to find the position under the mouse pointer.
- Loading fonts, and adapting to displays with a different scale.
- Drawing graphics on the screen, including text.

Canvas A Canvas is the surface that Views draw on during one frame. It is created by the Renderer at the start of the frame, then passed to different components, which use it to draw on the screen. The same Canvas is passed to every UIElement in the tree.

A Canvas always draws in the coordinates of the element that is currently being drawn, with the origin at the top left corner of that element. This is achieved by restricting the Canvas to a when an element starts to draw, and restoring the previous restriction when it finishes. Because of this, Views never need to know where their element is in the window. The Canvas provides the following operations:

- Restricting drawing to a subset of the canvas, and moving the origin to its top left corner. The previous restriction is restored afterwards.
- Drawing a line of text at a point, where parts of the text may have different colours.
- Drawing a filled rectangle (for example for selections and backgrounds).
- Drawing a line.

5.2.6 Extensions

PluginManager The PluginManager finds plugins and starts them. Responsible for loading plugins.

The start point of a plugin is executed once. It is expected to register the callbacks, commands, and configuration keys that the plugin provides, using the Plugin API, and then return. If the start point fails, the plugin is skipped, the error is reported to the user, and loading continues with the next plugin.

Plugin unloading or reloading may require the editor to restart.

PluginCatalog The PluginCatalog provides the list of plugins that can be installed and installs them. The catalog is a JSON array in a public Git repository; its address is fixed in the editor. Each entry has the fields of PluginInfo (see the Data model section) and `downloadUrl`, the HTTPS address of a ZIP archive that contains `info.json` and the codeFile. The catalog is downloaded in the background each time the Plugins view is opened and is kept only in memory. If it cannot be downloaded, only the installed plugins are shown.

When Install is pressed, the archive is downloaded and extracted into a temporary folder, its `info.json` is checked (a valid PluginInfo with the same `id` as the catalog entry, and an existing

codeFile), and the folder is moved to `plugins/<id>/`. The plugin is then installed and disabled. If any step fails, the temporary folder is deleted and an error message is shown. Enable and Disable add or remove the `id` in the `plugins` field of the Configuration, and the change takes effect after a restart. Create plugin creates `plugins/<id>/` with a default `info.json` and an empty codeFile and opens the codeFile in a new buffer.

ScriptRuntime The ScriptRuntime executes plugin code. It is an embedded interpreter or virtual machine which exposes the Plugin API to plugins and invokes plugin callbacks on behalf of the editor. The runtime must satisfy the following requirements:

- It can be embedded into the editor process.
- Errors raised by a plugin are caught and reported to the editor, and do not terminate the editor.
- The duration of a single plugin call can be limited. A call exceeding the limit is aborted and counted as a failure.
- Plugins are isolated from each other, so that they cannot modify each other's state without using the Plugin API.
- Plugins can access the editor only through the Plugin API.

Security consists of handling faulty code, rather than on stopping malicious actors from affecting users.

EditorService The EditorService provides a public programmatic interface to the editor. It is intended primarily for functionality which needs to manipulate the editor programmatically, such as a future scripting or extension system. The component should expose stable, high-level editor operations rather than exposing the internal implementation of components directly.

5.2.7 Infrastructure

EventBus The EventBus delivers notifications about changes in the editor to interested components. Core components publish events. Core components and plugins subscribe to the events they are interested in, plugins doing so through the Plugin API.

Examples of events include a file or buffer being opened, changed, saved, or closed, the active pane changing, and a configuration value changing. Events carry identifiers and a description of what changed, not references to internal objects.

Events are delivered on the editor's main thread. When an event occurs, the callbacks subscribed to it are called one after another. Events raised while a callback is running, for example because the callback edited a buffer, are queued and dispatched after the current callback returns. Callbacks are never nested.

IMPORTANT: For the EventBus to be able to receive events from components, event generation should be implemented within individual components (if relevant to the component, which is decided on a component-by-component basis).

Scheduler The Scheduler runs code at the times and on the thread chosen by the Application. It has two functions:

- Timers: running a callback after a delay or repeatedly. Used, for example, for the blinking of the cursor, for scrolling while the user drags a selection outside a Pane, and for timers requested by plugins.
- Results of background work: any thread may post a callback, which is then run on the main thread during a later iteration of the main loop. Posting a callback also wakes up the main loop if it is waiting.

Background work must not read or change editor state directly. It performs only the slow part (for example reading and decoding a file) and posts the result, which is applied by the main thread.

EncodingConverter Capable of detecting encoding of text, and is capable of converting between supported encodings.

FileController The FileController provides an abstraction over the operating system's filesystem. It is responsible for direct filesystem operations. Platform-specific filesystem functionality should be hidden behind its interface. Potential operations include:

- Creating, deleting, and renaming files.
- Reading and writing files.
- Creating and deleting directories.
- Querying file and directory information.
- Determining whether a file exists.
- Deducing/changing a file's text encoding. Done via EncodingConverter.
- When saving a Buffer to a file, convert from default encoding used in buffers, to the specific file encoding.

5.3 Data Store

The editor has to have the ability to interact with the file system in order to carry out its functions. Additionally, the editor is going to have certain data that has to persist across sessions. As an example, that would include global configurations, various histories (undo/redo, command history), installed plugins, etc. The editor keeps its own data under a single directory (or multiple directories) named after the code editor (i.e., *notepad-sharp*). The locations are OS-dependent:

- **POSIX** – the locations defined by the XDG Directory Specification (`https://specifications.freedesktop.org/basedir/latest/`).
- **Windows** – `%APPDATA%`, where the configuration, data and history directories below are sub-directories of the `editor` directory.

The data store has the following structure:

```
config/
  config.json              configuration file
plugins/
  <plugin-id>/
    info.json              PluginInfo of the installed plugin
    <codeFile>             plugin code
syntax/
  <language>.json          syntax of the language is described here
history/
  undo/
    <file-key>.json       stored EditHistory of one file
```

Each part is described below.

- **config/config.json**: the configuration file. It is the only file the user is expected to edit by hand or through the application interface.
- **plugins/**: one folder per installed plugin. The folder name equals the `id` in its `info.json`. A plugin is installed when its folder contains a valid `info.json`. The codeFile is the main code of the plugin that will contain its functionality and be executed.
- **syntax/**: each file language has it's own seperate file. The following example shows the structure of the file:

```
{
    "language": "mini",
    "extensions": [".min", ".m"],
    "rules": [
        { "match": "//.*$",                  "token": "comment" },
```

```
        { "match": "\"(\\\\.|[^\"])*\"",    "token": "string" },
        { "match": "\\b(if|else|while|return)\\b", "token": "keyword" },
        { "match": "\\b[0-9]+(\\.[0-9]+)?\\b",     "token": "number" }
    ]
}
```

- **history/undo/**: the stored EditHistory of Buffers, one file per saved buffer file. The file name is a unique key derived from the absolute path of the file. All data related to EditHistory is stored in the format described in the DataModel section.

The editor creates missing directories on startup. A missing configuration file is not an error: the editor starts with its built-in defaults. The plugin catalog is not part of the Data Store.

IMPORTANT: Edited files are not part of the Data Store. They are stored where the user saved them and are accessed through the FileController.

Configuration file

`config.json` is a JSON file with three optional sections. A missing section is the same as an empty one.

- `settings`: an object of setting keys and values (for example `"fontSize": 14`). The built-in keys are listed in the data model. Plugins may register more keys, and their names should start with the plugin identifier and a dot (for example `openai-chat.model`, this is enforced). The settings also contain `colors`, an object that maps a token group (keyword, comment, etc.) to a color. Colors are defined globally, which keeps the color scheme of the editor consistent, while the rules for recognizing the groups in text are defined separately for each language. Colors are given as hexadecimal RGB codes.

```
"colors": {
  "comment": "808080",
  "string": "00FF00",
  "keyword": "0000FF",
  "number": "00FF00"
}
```

- `keybinds`: a set of key value pairs, each with a `key` (for example `"Ctrl+Shift+P"`) and a `command` (a command identifier such as `buffer.save`). The defaults come from the CommandRegistry and are overridden by this list. If several entries use the same key combination, the last one is used.
- `plugins`: an ordered list of the enabled plugins, each with a unique `id`.

5.4 Dependencies

A dependency may fulfill multiple functions at the same time. There is also nothing stopping the implementing team from implementing the functionality themselves, if they deem it worthwhile. Here are functions that may eventually be handled by external dependencies:

- **GUI/Windowing.** Window creation, control, etc. (as an example of such dependencies, Qt(`https://en.wikipedia.org/wiki/Qt_(software)`, or GTK (`https://en.wikipedia.org/wiki/GTK`)).
- **Rendering.** For actually drawing stuff on screen.
- **Filesystem.** Instead of directly relying on an operating system.
- **Text processing (encoding handling included).**
- **RegEx.** For syntax rule definitions.
- **Search.** If in-editor search is implemented (for example, grep), then a dependency may be used for it.
- **Scripting/runtime.** Although the desginers were thinking about Lua, implementing team may use any other runtime if they wish, provided it is capable of carrying out the described functionality.
- **Networking.** Downloading the plugin catalog and plugin archives over HTTPS.
- **Archives.** Extracting the ZIP archives of plugins.

5.5 API

The Plugin API is the only way a plugin can read or change the editor. It is provided by the EditorService and is passed to the start point of every plugin, together with the parameters from the plugin's entry in the configuration. It is described in a language-neutral way, so the implementation may use any scripting language. The API is divided into groups of functions:

- **Buffers** – open, create, save and close buffers, and read or edit their text, including undo and redo.
- **Editor** – read and change the cursors and selections of the focused pane, and switch the buffer it shows.
- **Commands** – register new commands and execute built-in or plugin commands.
- **Settings** – register new setting keys and read or change them.
- **Events** – subscribe to notifications such as a buffer being opened, changed or saved.
- **Utilities** – show messages to the user, write to the log, use timers and access the clipboard.

The following rules apply to all calls. Plugins refer to buffers by identifier and never receive references to internal objects. All calls and callbacks run on the main thread, one at a time, and slow operations such as opening and saving files run in the background and report their result as an event. Names of plugin commands and settings must start with the plugin identifier and a dot, and a plugin can change only its own settings.

A plugin has a start point, which is called once and receives an `api` object with the groups of functions below. Everything a plugin does is a call on this object or a callback it gave to the editor.

```
function start(api, params: Map)  # params: values from the plugin's entry
                                  # in the configuration
```

Functions

```
# api.buffers
open(path: String) -> Integer    # returns the buffer id at once; the file
create() -> Integer              # new empty scratch buffer
save(id: Integer)                # background; result is reported by an event
close(id: Integer)               # fails if the buffer has unsaved changes
list() -> List<Integer>          # ids of all buffers
info(id: Integer) -> Map         # keys: filePath (or null), language, encoding, length,
                                 # lineCount, isModified, isLoading
getText(id: Integer, start: Integer, end: Integer) -> String
getLine(id: Integer, line: Integer) -> String
insert(id: Integer, position: Integer, text: String)
delete(id: Integer, start: Integer, end: Integer)
```

```
replace(id: Integer, start: Integer, end: Integer, text: String)
undo(id: Integer) -> Boolean     # false if there is nothing to undo
redo(id: Integer) -> Boolean

# api.editor    (acts on the focused pane and its editing context)
focusedBuffer() -> Integer
showBuffer(id: Integer)                 # switches the focused pane to another buffer
getCursors() -> List<Integer>           # positions
setCursors(positions: List<Integer>)    # at least one position
getSelections() -> List<List<Integer>>  # each item is [start, end]
setSelections(ranges: List<List<Integer>>)

# api.commands
register(id: String, title: String, handler: function(args: Map))
execute(id: String, args: Map)  # id of a built-in command (e.g. "buffer.save") or of a plugin

# api.settings
register(key: String, defaultValue: Value)   # only inside start()
get(key: String) -> Value               # any setting, built-in or from a plugin
set(key: String, value: Value)          # only keys registered by the same plugin

# api.events
subscribe(name: String, handler: function(payload: Map)) -> Integer   # returns a subscription
unsubscribe(subscription: Integer)

# api.ui, api.timers, api.clipboard
notify(message: String)           # short message shown to the user
log(message: String)
after(delayMs: Integer, callback: function) -> Integer   # also: every(intervalMs, callback),
getText() -> String               # system clipboard; also setText(text)
```

Rules

- A call with invalid arguments (unknown id, position outside the buffer, wrong type, and so on) fails with an error message and has no effect. How the failure is shown to the plugin (exception, error value) is decided by the language binding.
- All calls and callbacks run on the main thread, one at a time. `open` and `save` only start the work; the result arrives as an event. Reading or changing the text of a buffer that is still loading fails.
- Ids of plugin commands and keys of plugin settings must start with the plugin id and a dot (for example `openai-chat.ask`). A plugin can change only its own settings.
- Every call of plugin code by the editor (start point, command handler, event handler, timer)

has a time limit, as described in the ScriptRuntime section. Exceeding it, or raising an error, is a failure of that plugin only.

- Events are delivered as described in the EventBus section: on the main thread, in subscription order, never nested. An event is raised after the change has been completed.

Events

Every event handler receives a `Map` with the fields listed below. Fields are identifiers and values, never references to internal objects. Plugins must ignore fields they do not know.

```
editor.started        {}
                      # all plugins have been started
editor.closing        {}
buffer.opened         { bufferId, filePath, isLoading }
                      # filePath is null for a scratch buffer
buffer.loaded         { bufferId }
                      # the text of the buffer can now be read
buffer.changed        { bufferId, position, deletedText, insertedText }
                      # a text is null if empty
buffer.saved          { bufferId, filePath }
buffer.closed         { bufferId }
pane.focused          { bufferId }
cursors.changed       { bufferId, cursors, selections }
                      # of the focused editing context
config.changed        { key, oldValue, newValue }
command.executed      { commandId, success }
```

Failures of loading or saving a file are reported with the events `buffer.loadFailed` and `buffer.saveFailed`, both with the fields `bufferId` and `message`. Plugin-defined events and finer-grained events (for example per-pane and per-context events) are not part of this version of the API.

6 Low fidelity wireframes

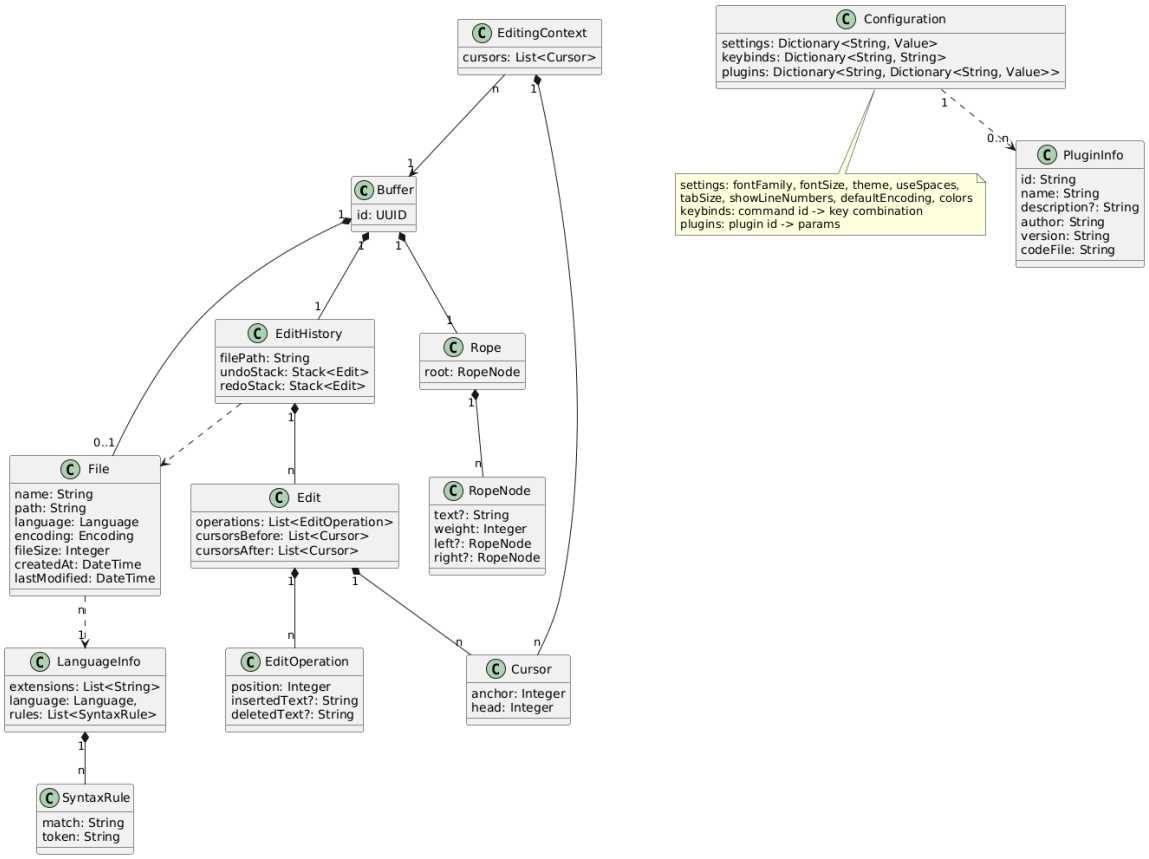
The main features of the application can be explored via this prototype.
A PDF version of it is available in the repository.

It lets to:

- Navigate between opened files (`main.py`, `notes.txt`)
- See possible editing options in `Edit` tab of the top horizontal menu
- See possible operations on files in `File` tab of the top horizontal menu
- Save a file via `File` tab of the top horizontal menu
- Navigate to plugins' section via `Plugins` tab of the top horizontal menu
- Navigate to settings' section via `Settings` tab of the top horizontal menu
- See a list of possible theme options in `Settings` section
- See a list of possible encoding options in `Settings` section
- Navigate to the main section via `Notepad#` tab of the top horizontal menu
- Split editor with an icon on the right side (under closing button)
- Close a file in the new text pane with a file closing button
- Access the opened files menu with `Ctrl+Tab` (use full screen mode)

7 Data model

The data model is used for determining the data that will be used inside the project. The data classes used in the project are shown in **3 figure.**. Each component has fields with the type of data that they take. Field values that could be **null** are shown with question marks. Some fields use enumerators which are not defined in the model but their suggested values are provided.



3 figure. *Data model*

7.1 Main components

The data model defines the information the editor stores and manages in order to meet these needs. It separates the content of a buffer, the history of changes made to it, and the user's configuration, so that each can be changed and persisted independently. The main components are:

- **Buffer** – represents a text buffer, which may optionally be tied to a file. It owns its content (Rope), its editing history (EditHistory) and its File, if it has one.
- **File** – stores descriptive information about the file a buffer is tied to: name, path, language, encoding, file size and timestamps. It is kept separate from the content so that file information can be changed or displayed without touching the text.
- **EditingContext** and **Cursor** – describe how the user interacts with an open buffer in one view (for example, one pane). The context holds a list of cursors, and each cursor is an anchor

and a head, so a selection is the range between them. Several contexts can refer to the same buffer, each with its own cursors. They are not stored on disk.

- **Configuration** – stores the user's configuration as key-value pairs in three sections: settings (font, theme, colors, etc.), keybinds and enabled plugins. New options can be added without changing the model. The configuration applies to all buffers.
- **PluginInfo** – describes a single plugin. It is stored as the `info.json` file in the plugin's folder and is used to list installed plugins.
- **EditHistory**, **Edit** and **EditOperation** – record the changes made by the user so that they can be undone and redone, and so that the history can persist between sessions. An Edit is one undo step and consists of one or more EditOperations (single insertions or deletions).
- **LanguageInfo** and **SyntaxRule** – the syntax highlighting definition of a language: its file extensions and an ordered list of rules. Each rule is a regular expression and the token type given to the text it matches.
- **Rope** and **RopeNode** – store the content of a buffer as a binary tree of text pieces. Each leaf node holds a fragment of text, while each internal node references a left and a right child and stores a weight, the total length of the text in its left subtree. The weight allows a position to be located by walking down the tree, so insertion and deletion in large files remain efficient, unlike with a single string that must be copied.

Buffer

The **Buffer** class represents a text buffer, which may optionally be tied to a file. It has three relations: a 1:1 relation to EditHistory, a 1:1 relation to Rope for the buffer content (which in turn has a 1:n relation to RopeNode) and a 1:1 relation to File (if the Buffer is a scratch Buffer, it's relation to File is null). All relations are unidirectional: the Buffer references the related objects, but they do not reference the Buffer. The Buffer owns its Rope, EditHistory and File, so deleting a Buffer also deletes them. It consists of the following fields:

- **id** – a unique identifier of the buffer. Takes a UUID value, generated automatically upon creation.

File

The **File** represents the file data which is tied to a buffer. It has a 1:1 relation to the Buffer model which points to the metadata of the specific file and it's configuration. It consists of the following fields:

- **name** – the name of the file. Takes in a string value from the file path.
- **path** – the location of the file on the file system. Takes in a string value. An example would be `"/desktop/project/"`. **Constraint: must we a valid path on the users computer**

- **language** – identifies the programming language of the document. Takes in a file extension string (for example, .py, .c). Buffers without a recognized language use no highlighting. **Constraint: must be one of the supported language types.**
- **encoding** – the character encoding used to read the buffer from and write it to its file. Takes in a string value corresponding to a supported encoding (e.g., `"UTF-8"`, `"UTF-16"`, `"ASCII"`). The default value is `"UTF-8"`. **Constraint: must be one of the supported encodings.**
- **fileSize** – the size of the file in bytes. **Constraint: must be a non-negative integer.**
- **createdAt** – the date and time when the file was created. Takes in a DateTime showing the year, month, day, hours, and minutes of the file's creation. **Constraint: must be a valid date time.**
- **lastModified** – the date and time when the file was last modified. Takes in a DateTime showing the year, month, day, hours, and minutes of the last modification. On creation it equals `createdAt`. **Constraint: must be a valid time that is not earlier than `createdAt`.**

EditingContext

The **EditingContext** class describes how the user interacts with an open buffer in one view, for example one pane. It stores only the cursors; the text itself stays in the Rope of the buffer. It is kept only in memory and is discarded when the view is closed. It has an n:1 relation to Buffer and an owned 1:n relation to Cursor. The relation to Buffer is unidirectional: several contexts can refer to the same Buffer (for example, a file open in two panes), each with its own cursors, but the Buffer does not know about them. Deleting a context also deletes its cursors.

An EditingContext never changes the text directly. All edits go through the Buffer it refers to: the Buffer applies the change to its Rope and records the corresponding Edit in its EditHistory. The context only decides which range is affected. It consists of the following field:

- **cursors** – the cursors of the view. Takes a list of Cursor objects. **Constraint:** the list contains at least one cursor; more than one means multiple cursors. When text is inserted or deleted, the positions after the change are shifted, in this and in every other EditingContext of the same Buffer.

Cursor

The **Cursor** class describes one cursor and its selection. It is owned by an EditingContext, or by an Edit when it is stored as a cursor state, and it has no relation to any other class. It refers to the text only by positions, so it does not depend on the structure of the Rope. The selected text is the range between `anchor` and `head`; if they are equal, nothing is selected. It consists of the following fields:

- **anchor** – the position where the selection started. Takes a non-negative integer. It stays in place while the selection is extended.

- **head** – the position of the cursor itself, where typed text is inserted. Takes a non-negative integer. It moves when the selection is extended, so it may be before or after the anchor. **Constraint:** `anchor` and `head` must not exceed the length of the content.

Configuration

The **Configuration** class is the global configuration of the editor. There is exactly one instance, which applies to all buffers, so it has no relationship to the Buffer class. It is the in-memory form of the configuration file in the Data Store: the Configuration Repository reads and validates the file at startup, keeps one Configuration object for the session, and provides values to the editor components (for example, the Pane). The configuration is stored as key-value pairs rather than fixed fields, so a new setting can be added without changing the model. Each field corresponds to one section of the file. The `plugins` field refers to **PluginInfo** by value (each key is the `id` of a PluginInfo), and the keys of `keybinds` are command identifiers of the CommandRegistry, which is not part of this data model. It consists of the following fields:

- **settings** – the editor settings. Takes a `Dictionary<String, Value>`, where the type of the value depends on the key. A key without an entry uses its default value. **Constraint:** each value must match the type and constraints of its key, otherwise the file is invalid. Unknown keys are ignored.
- **keybinds** – the keyboard shortcuts. Takes a `Dictionary<String, String>` that maps a key combination to a command identifier (e.g., `"Ctrl+Z": "undo"`). A command without an entry uses its default shortcut. **Constraints:** each command must be registered in the CommandRegistry. If same combination uses a different command, the one listed last is used.
- **plugins** – the enabled plugins. Takes a `Dictionary<String, Dictionary<String, Value>>` that maps a plugin identifier to the parameters passed to its start point. An empty dictionary means no plugins are loaded. **Constraint:** each key must be the `id` of an installed PluginInfo; otherwise the plugin is skipped and the error is reported.

The built-in keys of `settings` are listed below. Plugins can register additional keys, whose names should start with the plugin identifier (e.g., `openai-chat.fontSize`).

- **fontFamily** – the font of the editor text (e.g., `"Roboto Mono"`). **Constraint:** must be a value from the provided list.
- **fontSize** – the text size in pixels. Integer. **Constraint:** 6 to 64. **Default:** 14.
- **tabSize** – the number of spaces represented by a tab. Integer. **Constraint:** 1 to 64. **Default:** 4.
- **useSpaces** – whether spaces are used instead of tab characters. Boolean. **Default:** false.
- **showLineNumbers** – whether line numbers are displayed. Boolean. **Default:** true.

- **defaultEncoding** – the encoding assigned to new buffers (`"UTF-8"`, `"UTF-16"`). **Default:** `"UTF-8"`.
- **theme** – the visual theme (`"black on white"`, `"white on black"`). **Default:** `"black on white"`.
- **colors** – a dictionary that maps a token type (the `token` of a SyntaxRule) to a color, e.g. `{"comment": "808080", "keyword": "FFFFFF"}`. **Constraint:** each value is a color name or a 6-character hexadecimal string without #. A token type without an entry uses the default text color.

PluginInfo

The **PluginInfo** class describes a single plugin. It is stored as the file `info.json` in the plugin's folder, so the installed plugins can be listed. It does not reference any other class. It is referred to by value from Configuration, where each key of `plugins` is the `id` of a PluginInfo. It consists of the following fields:

- **id** – the unique identifier of the plugin (e.g., `"openai-chat"`). It is also the name of the plugin's folder. **Constraint:** may contain only lowercase letters, digits and hyphens, and must equal the name of the folder.
- **name** – the name of the plugin shown to the user (e.g., `"OpenAI Chat"`). Takes a string value.
- **description** – a short text explaining what the plugin does. Takes a string value. It is null when the author provides no description.
- **author** – the person or organization that created the plugin. Takes a string value.
- **version** – the version of the plugin in the form `major.minor.patch` (e.g., `"1.2.0"`), so that two versions can be compared.
- **codeFile** – the name of the file in the plugin's folder that contains the start point, which is executed when the plugin is loaded. Takes a string value. **Constraint:** the file must exist in the folder.

EditHistory

The **EditHistory** class contains the changes made by the user, so that they can be undone and redone with the undo and redo commands (whose shortcuts come from the keybinds). There is one EditHistory for each buffer, in a 1:1 relation from Buffer. The relationship is unidirectional, so the EditHistory does not reference its Buffer, and the BufferController passes the Buffer when an undo or redo is performed. When a Buffer is deleted, its EditHistory is also deleted. The history of a buffer that has a file is saved in persistent storage and restored when the file is opened again, so it

is matched to the file by its path, not by the id of the buffer. A buffer without a file has no stored history. It consists of the following fields:

- **filePath** – the absolute path of the file the history was recorded for. Takes a string value. It is the key that links a stored history to a file after a restart, and it equals the `path` of the File of the buffer.
- **undoStack** – the edits that can be reverted. A Stack of Edit objects, populated with push and reduced with pop. A new buffer has an empty stack.
- **redoStack** – the edits that have been undone and can be applied again. A Stack of Edit objects. It is empty for a new buffer and is cleared whenever a new change is made.

Edit

The **Edit** class is one undo step. It groups the changes that are undone or redone together, for example the edits of all cursors in a multicursor edit, or a run of typed characters. There is a 1:n relation from EditHistory to Edit, and an owned 1:n relation from Edit to EditOperation and to Cursor. The relations are unidirectional, and deleting an EditHistory deletes its Edits. It consists of the following fields:

- **operations** – the insertions and deletions of this step. Takes an ordered list of EditOperation objects, applied in the listed order and reverted in reverse order. **Constraint:** must contain at least one operation.
- **cursorsBefore** – the cursors before the edit, restored by undo. Takes a list of Cursor objects.
- **cursorsAfter** – the cursors after the edit, restored by redo. Takes a list of Cursor objects.

EditOperation

The **EditOperation** class is a single insertion or deletion of text. It is owned by one Edit and has no relation to any other class. It consists of the following fields:

- **position** – the position in the buffer where the change starts. Takes a non-negative integer. **Constraint:** must not be greater than the length of the buffer at the moment the operation is applied.
- **insertedText** – the text that was inserted. It is null for a deletion.
- **deletedText** – the text that was deleted, kept so that the deletion can be undone. It is null for an insertion. **Constraint:** exactly one of `insertedText` and `deletedText` must be non-null and non-empty.

LanguageInfo

The **LanguageInfo** class is the syntax highlighting definition of a language. Definitions are stored as JSON files, are read-only, and are loaded at startup. A LanguageInfo is not tied to a buffer: the `language` field of the File of a buffer identifies the definition to use, so many buffers can share one definition. The relation of File to LanguageInfo is n:1 and unidirectional. LanguageInfo has an owned 1:n relation to SyntaxRule. The colors of token types are not part of the definition but of the `colors` setting of Configuration. It consists of the following fields:

- **language** – the language that the definition describes. Takes a value of the **Language** enumerator which contains existing languages. **Constraint:** must be unique among all loaded definitions. `PLAIN_TEXT` has no rules.
- **extensions** – the file extensions of files written in this language (e.g., `[".py"]`). Takes a list of strings. It is used to choose the language when a file is opened. An empty list means the language is never chosen automatically.
- **rules** – the rules used to tokenize the text. Takes an ordered list of SyntaxRule objects. The order is the priority: a character claimed by an earlier rule is not matched by a later one.

SyntaxRule

The **SyntaxRule** class is a single rule of a language definition. It is owned by one LanguageInfo, has no relation to any other class, and is applied to one line of text at a time. It consists of the following fields:

- **match** – the regular expression that finds the tokens. Takes in a string value. **Constraint:** must be a valid regular expression that does not match an empty string. An invalid rule is skipped and the error is reported.
- **token** – the type given to the matched text, for example `comment`, `string`, `keyword` or `number`. Takes in a string value. It is the key used to find the color in the `colors` setting; a type without a color is drawn in the default text color.

Rope

The **Rope** class is used to store the text of a buffer. It wraps a tree of **RopeNode** objects and provides access to the tree through its root node. There is a 1:n relationship from Rope to RopeNode: one Rope consists of many RopeNodes. The relationship is unidirectional: the Rope references its root, and the nodes reference only their children. When a Rope is deleted, its RopeNodes are also deleted. It consists of the following field:

- **root** – a reference to the root **RopeNode** of the tree. All other nodes are reachable from the root. An empty buffer has a root that is a leaf node with an empty text.

RopeNode

The **RopeNode** class represents a single node in the tree of a **Rope**. By default, all text is stored as UTF-8, with LF (line feed, or $\backslash n$ as opposed to Windows's carriage return line feed, or $\backslash r \backslash n$) representing newlines. A node is either a leaf, which stores a bounded-size piece of the buffer text, or an internal node, which connects to two child nodes. RopeNode is used for storing a piece of text and so that deleting updating, inserting text would involve simply creating, updating or inserting a new RopeNode. The relationship to the children is unidirectional: a node references its children but not its parent. It consists of the following fields:

- **text** – the text stored in the node. Takes in a string value. It is null (or empty) in internal nodes and is set in leaf nodes. **Constraint: its length must not exceed the maximum node size.**
- **weight** – the total length of the text in the left subtree of the node (or the length of `text` if the node is a leaf). It is used to determine positions within the Rope (counted in Unicode code points). Takes in a non-negative integer.
- **left** – a reference to the left child **RopeNode**. It is null in leaf nodes.
- **right** – a reference to the right child **RopeNode**. It is null in leaf nodes. **Constraint: `left` and `right` are either both null (leaf node) or both non-null (internal node).**